Jobsheet 2

Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom

Disusun oleh:
Mukhammad Raffi Zabra

T1 – 1G

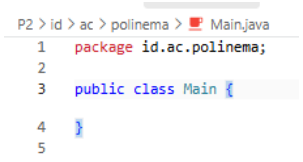
254107020059

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141

TAHUN 2026-2027

Langkah 1: Melanjutkan Proyek bank-mini

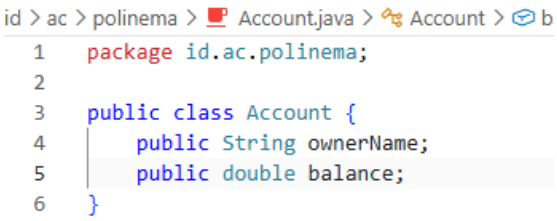


Gambar 1 membuat class main

Penjelasan : langkah ini menyiapkan struktur awal proyek dan membuat class main yang berisi method main, method main ini sebagai awal eksekusi program

Langkah 2: Kelas Account Minimal dan Objek Pertama

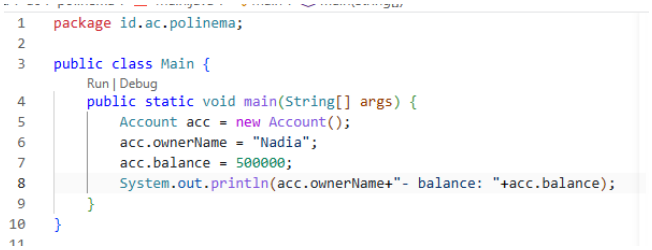
Class account



Gambar 2 membuat class account dan atributnya

Penjelasan : di sini kita membuat sebuah kelas bernama Account yang fungsinya sebagai cetakan atau rancangan awal rekening bank. Saat ini kelasnya baru berisi atribut, yaitu ownerName untuk menyimpan nama pemilik dan balance untuk menyimpan jumlah saldo

class main



Gambar 3 membuata project account dan isi atributnya

penjelasan : Pada kelas utama ini, kita mencoba mencetak satu rekening nyata bernama acc dari cetakan Account menggunakan perintah new. Karena belum ada sistem pengisian otomatis, nama "Nadia" dan saldo 500000 diisikan secara manual satu per satu ke dalam atributnya sebelum dicetak

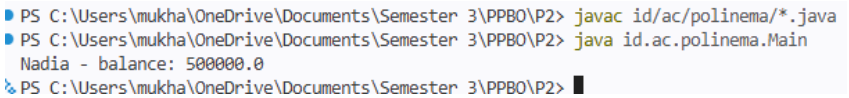
checkpoint

Jalankan proyek (klik kanan proyek > Run, atau tekan tombol F6). Apabila menggunakan terminal:
`javac -d out src/id/ac/polinema/*.java && java -cp out id.ac.polinema.Main`.

✅ **Checkpoint:** program menampilkan `Nadia - balance: 500000.0`.

⚠ **Jika gagal:** apabila muncul `error: cannot find symbol`, periksa apakah nama berkas sama persis dengan nama kelasnya (`Account.java` untuk `class Account`). Java bersifat case-sensitive, sehingga huruf besar dan kecil harus sesuai.

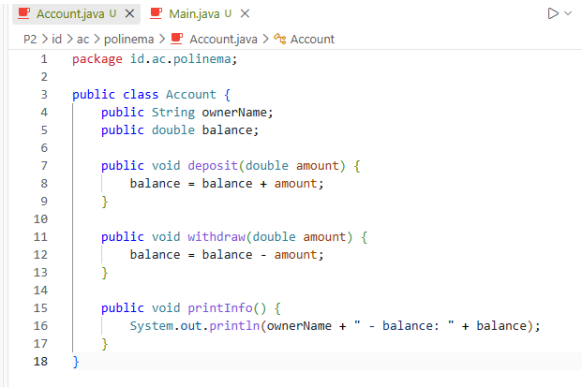
hasil :



Gambar 4 hasil dari langkah 2

penjelasan : Hasil eksekusi terminal menampilkan Nadia - balance: 500000.0. Ini membuktikan objek berhasil terbuat

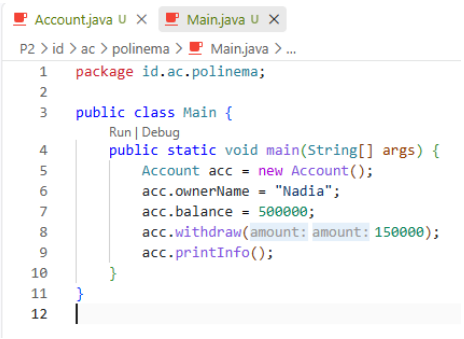
Langkah 3: Menambahkan Method (Perilaku)
Class account



Gambar 5 menambahkan method di class account

Penjelasan : Pada tahap ini, kelas Account diberi fungsi aksi atau method. Ada fungsi deposit untuk menambah tabungan, withdraw untuk memotong saldo, dan printInfo untuk merapikan proses cetak data rekening ke layar.

Class main



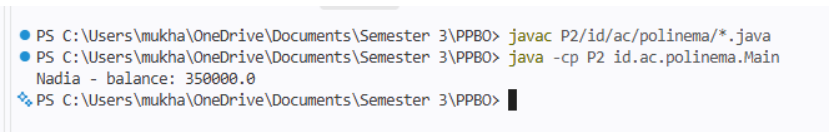
Gambar 6 di dalam class main kita gunakan methodnya

penjelasan : Di file utama, kita tidak lagi mencetak data saldo secara manual. Kita langsung memerintahkan objek acc untuk menyetor uang, mengambil uang lewat method withdraw(150000), lalu memanggil method printInfo() agar objek menampilkan sendiri datanya.

Jalankan ulang.

> ✅ **Checkpoint:** program menampilkan `Nadia - balance: 350000.0`.
>
> ⚠ **Jika gagal:** apabila muncul galat `non-static method deposit(double) cannot be referenced from a static context`, method dipanggil tanpa melalui objek (misalnya `Account.deposit(500000)`). Method non-static wajib dipanggil melalui objeknya, yaitu `acc.deposit(500000)`.

Hasil :



Gambar 7 hasil langkah 3

Penjelasan : Terminal menampilkan Nadia - balance: 350000.0. Angka ini membuktikan method withdraw bekerja dengan benar memotong saldo awal 500.000 menjadi 350.000.

Langkah 4: Konstruktor
class account



Gambar 8 tambah konstruktor di class account

penjelasan : Di sini ditambahkan sebuah konstruktor dengan parameter nama dan saldo. Tujuannya supaya saat sebuah rekening baru dibuat, nama dan saldonya wajib langsung diisi sejak awal sehingga data tidak ada yang kosong atau terlewat. Kata this dipakai untuk membedakan antara variabel milik kelas dengan kiriman data dari luar.

class main

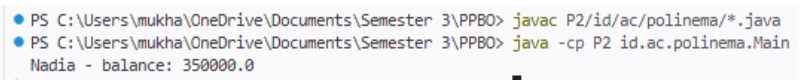


Gambar 9 pakai kontruktor di class main

penjelasan : Di file Main jadi jauh lebih rapi dan ringkas. Kita bisa langsung memasukkan nama "Nadia" dan saldo 500.000 tepat di dalam tanda kurung saat memanggil new Account.

> ✅ **Checkpoint:** program menampilkan `Nadia - balance: 350000.0`, output sama persis dengan Langkah 3, kode `Main.java` kini jauh lebih ringkas.
>
> ⚠ **Jika gagal:** apabila muncul galat `constructor Account in class Account cannot be applied to given types`, periksa apakah jumlah dan urutan argumen pada `new Account(...)` sudah sesuai dengan parameter konstruktornya.

hasil :



Gambar 10 hasil langkah 4

penjelasan : Output terminal tetap menghasilkan Nadia - balance: 350000.0. Hasilnya sama persis seperti sebelumnya, tetapi proses penyiapan data di dalam kodenya kini jauh lebih aman dan cepat

Langkah 5: Method dengan Logika dan Nilai Kembali
Class account



Gambar 11 menambahkan method baru di class account

Penjelasan : Di sini ditambahkan fungsi pengecekan logika. Fungsi formatBalance bertugas memberi tanda pemisah ribuan pada angka saldo, sedangkan isOverdrawn mengecek apakah saldo bernilai minus. Method withdraw juga diperbarui agar transaksi otomatis dibatalkan jika saldo tidak mencukupi saat ditarik.

Class main



Gambar 12 di class main mencetak saldo terformat dan penarikan saldo

Penjelasan : Di file ini kita menguji fitur barunya. Kita mencoba mencetak saldo dengan tampilan format ribuan, lalu sengaja menarik uang dalam jumlah besar (1.000.000) yang melebihi batas saldo yang tersisa.

> ✅ **Checkpoint:** program mencetak empat baris: `Nadia - balance: 350000.0`, `Formatted: 350,000.00`, `Withdrawal rejected: insufficient balance.`, lalu `Nadia - balance: 350000.0` lagi (saldo kembali seperti semula karena penarikan kedua ditolak).

> ⚠ **Jika gagal:** apabila hasil `formatBalance()` tidak menampilkan pemisah ribuan, periksa kembali format string `"%,.2f"`: tanda koma sebelum `.2f` yang mengaktifkan pemisah ribuan.

> **Catatan.** Cara `withdraw()` menolak penarikan di atas cukup janggal: mengurangi saldo dulu, memeriksa lewat `isOverdrawn()`, baru membatalkan bila ternyata salah. Cara yang lebih bersih adalah memeriksa kecukupan saldo SEBELUM saldo diubah sama sekali, itulah yang dilakukan Pertemuan 3. Lebih penting lagi, validasi ini bisa dilewati sepenuhnya: karena `balance` masih atribut publik, kode lain bebas menulis `acc.balance = -999999;` secara langsung, tanpa pernah melalui `withdraw()` atau `isOverdrawn()` sama sekali. Encapsulation pada Pertemuan 3 menutup celah ini: begitu `balance` menjadi privat, satu-satunya jalan mengubahnya adalah lewat method yang sudah divalidasi.

hasil :

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> javac P2/id/ac/polinema/*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> java -cp P2 id.ac.polinema.Main
Nadia - balance: 350000.0
Formatted: 350.000,00
Withdraw rejected : insufficient balance.
Nadia - balance: 350000.0
```

Gambar 13 hasil langkah 5

penjelasan : Terminal menunjukkan saldo terformat rapi menjadi 350.000,00. Selain itu, muncul pesan penolakan Withdraw rejected : insufficient balance dan saldo tetap utuh di angka 350.000 karena sistem berhasil membatalkan penarikan berlebih tersebut.

Langkah 6: Referensi, Aliasing, dan null
Class main

```
Account.java U   Main.java 1, U
P2 > id > ac > polinema > Main.java > Main > main(String[])
1   package id.ac.polinema;
2
3   public class Main {
        Run | Debug
4       public static void main(String[] args) {
5           Account original = new Account(ownerName: "Nadia", balance: 500000);
6           Account copy = original;
7           copy.deposit(amount: 100000);
8
9           System.out.println("Via original: "+original.balance);
10          System.out.println("Via copy: "+copy.balance);
11
12          Account empty = null;
13          empty.printInfo();    Null pointer access: The variable empty can onl
14      }
15  }
16
```

Gambar 14 di class main membuat 2 variabel

Penjelasan : Di kodingan ini kita membuat dua pengujian. Pertama membuat variabel copy yang disamakan dengan original, lalu menyetor uang lewat copy. Kedua, kita sengaja membuat variabel empty bernilai null lalu memanggil fungsinya.

Jalankan.

> ✅ **Checkpoint:** kedua baris pertama (`Via original:` dan `Via copy:`) sama-sama menampilkan nilai `600000.0`, meskipun hanya copy yang menerima deposit kedua. Hal ini bukan merupakan galat, melainkan cara kerja referensi: original dan copy menunjuk ke objek yang sama persis, sehingga perubahan melalui salah satu variabel otomatis terlihat pada variabel lainnya. Baris berikutnya akan menghasilkan `NullPointerException`, dan hal ini memang disengaja.

Berikut ilustrasinya pada stack dan heap:

Perbaiki dengan menghapus baris terakhir (`Account empty = null;` beserta pemanggilan `printInfo()` di atasnya) agar program dapat berjalan kembali tanpa galat:

> ⚠ **Jika gagal:** `NullPointerException` selalu muncul apabila method dipanggil pada referensi yang belum menunjuk ke objek mana pun (`null`). Solusinya selalu sama, yaitu memastikan objek telah benar-benar dibuat dengan new sebelum method-nya dipanggil.

hasil :

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> javac P2/id/ac/polinema/*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> java -cp P2 id.ac.polinema.Main
Via original: 600000.0
Via copy: 600000.0
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "id.ac.polinema.Account.printInfo()"
because "<local3>" is null
        at id.ac.polinema.Main.main(Main.java:13)
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO>
```

Gambar 15 hasil langkah 6

penjelasan : Terminal menampilkan saldo original dan copy yang sama-sama naik menjadi 600.000 karena keduanya sebenarnya menunjuk ke satu fisik rekening yang sama di memori. Setelah itu muncul error merah NullPointerException, yang sengaja dimunculkan untuk membuktikan bahwa variabel kosong tidak bisa disuruh menjalankan perintah apa pun.

Langkah 7: Array of Objects, Banyak Objek dari Satu Kelas
Class main

Account.java U × Main.java U ×

P2 > id > ac > polinema > Main.java > Main > main(String[])

```java
package id.ac.polinema;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Account[] accounts = new Account[3];
        accounts[0] = new Account(ownerName: "Nadia", balance: 500000);
        accounts[0].withdraw(amount: 150000);

        accounts[1] = new Account(ownerName: "Budi", balance: 1000000);

        accounts[2] = new Account(ownerName: "Sari", balance: 750000);
        accounts[2].withdraw(amount: 250000);

        for (Account acc : accounts){
            acc.printInfo();
        }
    }
}

```

Gambar 16 membuat array account di class main

Penjelasan : Kodingan ini menunjukkan cara mengumpulkan banyak rekening sekaligus ke dalam satu wadah daftar atau array bernama accounts berkapasitas 3 slot. Tiap slot diisi rekening berbeda (Nadia, Budi, Sari), lalu semua datanya dicetak berurutan menggunakan perulangan ringkas for-each

> ⚠ **Jika gagal:** `ArrayIndexOutOfBoundsException` menunjukkan bahwa indeks yang diisi tidak tersedia, misalnya `accounts[3]` padahal array tersebut hanya berukuran 3 (indeks yang valid: 0, 1, 2).

> ✅ **Checkpoint:** program mencetak tiga baris - `balance:` (satu per elemen array, dengan nilai yang berbeda-beda karena setiap `Account` memiliki datanya sendiri): `Nadia - balance: 350000.0`, `Budi - balance: 1000000.0`, `Sari - balance: 500000.0`.

Hasil :

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> javac P2/id/ac/polinema/*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> java -cp P2 id.ac.polinema.Main
Nadia - balance: 350000.0
Budi - balance: 1000000.0
Sari - balance: 500000.0
```

Gambar 17 hasil langkah 7

Penjelasan : Terminal langsung menampilkan data ketiga rekening secara berurutan. Saldo Nadia berkurang jadi 350.000, Budi tetap 1.000.000, dan Sari berkurang jadi 500.000, yang membuktikan tiap data di dalam array memiliki catatan saldonya masing-masing tanpa tertukar satu sama lain

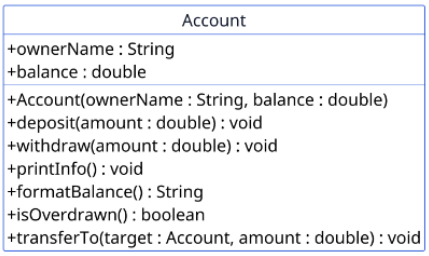
D. Tugas dan Deliverable
Screenshot output program setelah Langkah 7

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> javac P2/id/ac/polinema/*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> java -cp P2 id.ac.polinema.Main
Nadia - balance: 350000.0
Budi - balance: 1000000.0
Sari - balance: 500000.0
```

Gambar 18 hasil screenshot langkah 7

Tugas mandiri:

1. Bank Mini perlu memindahkan saldo antar rekening. Tambahkan method `transferTo` ke `Account` sesuai diagram kelas UML berikut. Tidak ada contoh kode untuk langkah ini, rancang sendiri isinya berdasarkan method yang sudah tersedia (`deposit()`, `withdraw()`) pada diagram:



Class account



```java
package id.ac.polinema;

public class Account {
    public String ownerName;
    public double balance;

    public Account(String ownerName, double balance) {
        this.ownerName = ownerName;
        this.balance = balance;
    }

    public void deposit(double amount) {
        balance = balance + amount;
    }

    public void withdraw(double amount) {
        balance = balance - amount;
        if (isOverdrawn()) {
            balance = balance + amount;
            System.out.println("Withdraw rejected : insufficient balance.");
        }
    }

    public void printInfo() {
        System.out.println(ownerName + " - balance: " + balance);
    }

    public String formatBalance() {
        return String.format("%,.2f", balance);
    }

    public boolean isOverdrawn() {
        return balance < 0;
    }

    public void transferTo(Account target, double amount) {
        this.withdraw(amount);
        if (!this.isOverdrawn()) {
            target.deposit(amount);
        }
    }
}
```

Gambar 19 menambahkan method transferTo di class account

Penjelasan : Method transferTo ditambahkan dengan dua parameter: target dan amount. Jadi logikanya mengecek apakah saldo cukup. Jika cukup, maka method withdraw() dijalankan pada akun pengirim, dan method deposit() dijalankan pada akun target. Jika tidak cukup, maka hanya memanggil withdraw() agar tercetak pesan penolakan penarikan.

Class main



```java
package id.ac.polinema;

public class Main {
    Run | Debug
    public static void main(String[] args) {

        Account from = new Account(ownerName: "Rappi", balance: 5000000);
        Account to = new Account(ownerName: "Rafid", balance: 2500000);

        from.transferTo(to, amount: 1500000);

        from.printInfo();
        to.printInfo();
    }
}
```

Gambar 20 pengujian method transferTo pada class main

Penjelasan : Di class Main melakukan pengujian dengan menginstansiasi dua objek Account, yaitu "Rappi" dengan saldo 5.000.000 dan "Rafid" dengan saldo 2.500.000. lalu method from.transferTo(to, 1500000) dipanggil untuk memindahkan saldo sebesar 1.500.000 dari rekening Rappi ke rekening Rafid.

Hasil :

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> javac P2/id/ac/polinema/*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> java -cp P2 id.ac.polinema.Main
Rappi - balance: 3500000.0
Rafid - balance: 4000000.0
```

Gambar 21 hasil dari tugas mandiri

2. Jawab secara singkat (2-3 kalimat untuk masing-masing pertanyaan):
(a) apa perbedaan antara objek dengan referensi ke objek?
Jawaban : Objek adalah wujud atau data nyata hasil instansiasi suatu kelas yang tersimpan di dalam memori Heap beserta nilai atributnya. Sedangkan referensi ke objek adalah variabel yang tersimpan di memori Stack yang hanya berfungsi memegang alamat memori untuk menunjuk ke letak objek tersebut. Jadi, referensi hanyalah penunjuk, sementara objek adalah fisik datanya.

(b) kapan tepatnya konstruktor sebuah kelas dijalankan?
Jawaban : Konstruktor dijalankan secara otomatis tepat saat proses instansiasi objek baru dilakukan. Jadi ketika kita memanggil kata kunci new yang diikuti oleh nama kelasnya, seperti pada pemanggilan new Account(...). Tujuannya adalah langsung menginisialisasi atribut agar data objek lengkap sejak awal dibuat.

HASIL DAN PEMBAHASAN

Praktikum Jobsheet 2 berhasil mengimplementasikan konsep dasar PBO melalui pembuatan kelas Account pada proyek Bank Mini. Kelas ini menggabungkan atribut data (ownerName, balance) dengan method aksi (deposit, withdraw, printInfo) untuk memproses saldo rekening. Penambahan konstruktor memastikan data objek langsung terisi lengkap saat perintah new dijalankan. Praktikum ini juga membuktikan sifat referensi di memori Java. Variabel objek hanya menyimpan alamat penunjuk di stack, sedangkan data aslinya berada di heap. Hal ini terbukti dari efek *aliasing* saat menyalin variabel, munculnya galat *NullPointerException* ketika memanggil referensi kosong, serta kemudahan mengelola banyak akun sekaligus menggunakan *Array of Objects*. Pada tugas akhir, method transferTo berhasil menghubungkan interaksi antar-objek untuk memindahkan saldo dari rekening pengirim ke penerima.

KESIMPULAN

1. Kelas berfungsi sebagai cetakan yang menyatukan atribut dan method, sedangkan objek adalah wujud nyata hasil cetakan tersebut di memori.
2. Variabel objek di Java bekerja sebagai referensi (penunjuk alamat), sehingga pemahaman alokasi memori sangat penting agar terhindar dari *aliasing* yang tidak disengaja maupun galat *NullPointerException*.
3. Penggunaan konstruktor menjamin data objek selalu siap pakai sejak awal, dan interaksi antar-objek mempermudah pemodelan alur kerja sistem seperti transaksi perbankan.

Lampiran

Link github repositori : https://github.com/Rappizr/PPBO/tree/main/P2/id/ac/polinema